MI.2030 pseudokood ↔ Python spikker

Nov 3, 2026 · @Hell

Pseudokoodi kokkulepped

Pseudokood kirjeldab algoritmi eesti keeles ja ilma Pythoni süntaksita; Pythonis vastab igale võtmesõnale üks konstruktsioon. Taane on mõlemas sama: plokk algab koolonist (Python) või võtmesõnast (pseudokood) ja lõpeb taande lõppedes. Pseudokoodis kirjutame plokivõtmesõnad SUURTE TÄHTEDEGA ja omistuse noolega `←`, et seda ei aetaks segi võrdlusega.

Pseudokood	Python	Tähendus
`SISESTA x`	`x = input("...")`	loe kasutajalt sõne
`VÄLJASTA x`	`print(x)`	näita ekraanil
`x ← avaldis`	`x = avaldis`	omistus
`KUI t SIIS … MUIDU KUI u SIIS … MUIDU … LÕPP KUI`	`if t:` / `elif u:` / `else:`	hargnemine
`KUNI t KORDA … LÕPP KUNI`	`while t:`	eelkontrolliga tsükkel
`IGA e KOGUMIS k KORDA …`	`for e in k:`	läbi kõik elemendid
`KORDA i = 1 … n`	`for i in range(1, n + 1):`	loendustsükkel (NB! `range` ülemine piir ei kuulu vahemikku)
`KATKESTA` / `JÄTKA`	`break` / `continue`	välju tsüklist / järgmine samm
`FUNKTSIOON f(p) … TAGASTA v`	`def f(p):` … `return v`	alamprogramm
`LOE FAILIST f` / `KIRJUTA FAILI f`	`with open(f, encoding="utf-8") as fail:`	andmevahetus failiga

Pseudokood	Python	Tähendus
`PROOVI … VEA KORRAL …`	`try:` … `except ValueError:`	erindi käsitlemine
`KONTROLLI t`	`assert t`	eeldus / test

R (reegel): Pythonis on taane süntaksi osa. K (konventsioon): taane on 4 tühikut, muutujanimed `vaiketahed_alakriipsuga`, konstandid `SUURTE_TAHTEDEGA`.

ÕV1 – Baaskonstruktsioonid programmilõiguna

Oskab programmeerimise baaskonstruktsioone (hargnemine, tsükkel, alamprogramm) esitada programmilõikudena.

Iga algoritm koosneb neljast ehitusplokist: jada, hargnemine, kordus ja alamprogramm. Allpool on igal üks pseudokood ja sama Python-lõik kaupluse hinnaarvestuse näitel.

1. Jada – laused täidetakse järjest

```
hind ← 12.50
kogus ← 3
summa ← hind * kogus
VÄLJASTA summa
```

```
hind = 12.50
kogus = 3
summa = hind * kogus
print(summa)
```

2. Hargnemine – üks-, kahe- ja mitmeharuline

```
KUI summa >= 100 SIIS
    allahindlus ← 0.10
MUIDU KUI summa >= 50 SIIS
    allahindlus ← 0.05
MUIDU
    allahindlus ← 0
LÕPP KUI
```

```
if summa >= 100:
    allahindlus = 0.10
elif summa >= 50:
    allahindlus = 0.05
else:
    allahindlus = 0
```

R: harusid kontrollitakse ülevalt alla ja täidetakse ainult esimene tõene haru. Seega järjesta tingimused kitsamast laiemani.

3. Kordus – eelkontrolliga ja loendustsükkel

Kui korduste arv pole ette teada, kasuta `KUNI` / `while`:

```
summa ← 0
SISESTA hind
KUNI hind ≠ "" KORDA
    summa ← summa + arv(hind)
    SISESTA hind
LÕPP KUNI
```

```
summa = 0
hind = input("Hind (tühi = lõpp): ")
while hind != "":
    summa = summa + float(hind)
    hind = input("Hind (tühi = lõpp): ")
```

Kui kogum või korduste arv on teada, kasuta `IGA` / `for`:

```
summa ← 0
IGA hind KOGUMIS hinnad KORDA
    summa ← summa + hind
LÕPP IGA
```

```
summa = 0
for hind in hinnad:
    summa += hind
```

4. Alamprogramm – nimega korduvkasutatav plokk

```
FUNKTSIOON hind_kaibemaksuga(netohind, maar)
    TAGASTA netohind * (1 + maar)
LÕPP FUNKTSIOON

VÄLJASTA hind_kaibemaksuga(10, 0.24)
```

```
KAIBEMAKSU_MAAR = 0.24

def hind_kaibemaksuga(netohind, maar=KAIBEMAKSU_MAAR):
    """Tagastab hinna koos käibemaksuga."""
    return netohind * (1 + maar)

print(hind_kaibemaksuga(10))
```

R: `return` lõpetab funktsiooni kohe; funktsioon ilma `return`-ita tagastab `None`. K: funktsioon teeb üht asja ja tema nimi ütleb, mida.

ÕV2 – Tekstülesandest programmini

Oskab tekstina püstitatud lihtsa ülesande realiseerida arvutiprogrammina.

Enne koodi kirjuta ülesandest välja sisend, töötlus ja väljund; pseudokood tuleb sellest tabelist peaaegu ise.

Näidisülesanne. Kasutaja sisestab toote netohinna ja koguse. Programm arvutab summa koos 24% käibemaksuga. Kui summa on üle 100 €, antakse 10% allahindlust. Tulemus kuvatakse kahe komakohaga. Vigase sisendi korral palutakse uuesti sisestada.

1. **Leia nimisõnad → muutujad:** netohind, kogus, summa, allahindlus.
2. **Leia arvud → nimega konstandid:** 24% → `KAIBEMAKSU_MAAR`, 100 € → `ALLAHINDLUSE_PIIR`, 10% → `ALLAHINDLUS`.
3. **Leia "kui" → hargnemine,** "uuesti" / "iga" → tsükkel, korduv arvutus → funktsioon.
4. **Kirjuta käsitsi 2–3 näidet** (sh piirjuht 100 €), siis pseudokood, siis Python.

Sisend	Töötlus	Väljund
netohind (float ⩾ 0), kogus (int ⩾ 1)	summa = netohind × kogus × 1,24; kui summa > 100, siis × 0,9	summa kahe komakohaga

```
FUNKTSIOON loe_arv(küsimus)
    KORDA LÕPMATULT
        SISESTA tekst
        PROOVI
            TAGASTA arv(tekst)
        VEA KORRAL
            VÄLJASTA "Vigane arv, proovi uuesti"

netohind ← loe_arv("Netohind")
kogus ← loe_arv("Kogus")
summa ← netohind * kogus * (1 + KAIBEMAKSU_MAAR)
KUI summa > ALLAHINDLUSE_PIIR SIIS
    summa ← summa * (1 - ALLAHINDLUS)
VÄLJASTA summa kahe komakohaga
```

```
KAIBEMAKSU_MAAR = 0.24
ALLAHINDLUSE_PIIR = 100
ALLAHINDLUS = 0.10


def loe_arv(kusimus):
    while True:
        tekst = input(f"{kusimus}: ").replace(",", ".")
        try:
            return float(tekst)
        except ValueError:
            print("Vigane arv, proovi uuesti")


def arvuta_summa(netohind, kogus):
    summa = netohind * kogus * (1 + KAIBEMAKSU_MAAR)
    if summa > ALLAHINDLUSE_PIIR:
        summa *= 1 - ALLAHINDLUS
    return summa


netohind = loe_arv("Netohind")
kogus = int(loe_arv("Kogus"))
print(f"Summa: {arvuta_summa(netohind, kogus):.2f} €")
```

NB! Eesti kasutaja kirjutab komaga (`12,50`); `.replace(",", ".")` teeb sellest Pythonile sobiva `12.50` . Arvutus on eraldi funktsioonis, et seda saaks hiljem pytestiga testida ilma `input` -ita.

ÕV3 – Põhikonstruktsioonid ja andmevahetus

Oskab kasutada põhilisi programmeerimiskonstruktsioone: muutuja, avaldis, omistuslause, tingimuslause, tsükkel, alamprogramm, andmevahetus kasutaja ja failidega.

Muutuja, avaldis, omistus

```
kogus ← 3
kogus ← kogus + 1
a, b ← b, a          // vaheta väärtused
```

```
kogus = 3
kogus += 1            # sama mis kogus = kogus + 1
a, b = b, a           # mitmene omistus
```

Tehete järjekord (kõrgemast)	Python	Näide
sulud	`( )`	`(2 + 3) * 4` → 20
astendamine	`**`	`2 ** 3` → 8
korrutamine, jagamine	`* / // %`	`7 // 2` → 3, `7 % 2` → 1
liitmine, lahutamine	`+ -`	`2 + 3 * 4` → 14
võrdlemine	`< <= == != >= >`	`0 <= x < 10`
loogika	`not` , siis `and` , siis `or`	`not a or b` = `(not a) or b`

Tingimuslause loogiliste tehetega

```
KUI vanus >= 18 JA on_klient SIIS
    VÄLJASTA "Soodushind"
LÕPP KUI
```

```
if vanus >= 18 and on_klient:
    print("Soodushind")
```

Piirjuhud, mida alati kontrollida: võrdus piiril (`>` vs `>=`), 0, negatiivne arv, tühi sõne, tühi järjend.

Tsükkel katkestamise ja indeksiga

```
IGA (nr, toode) KOGUMIS ostukorv KORDA
    KUI toode = "" SIIS JÄTKA
    KUI nr > 10 SIIS KATKESTA
    VÄLJASTA nr, toode
LÕPP IGA
```

```
for nr, toode in enumerate(ostukorv, start=1):
    if toode == "":
        continue
    if nr > 10:
        break
    print(nr, toode)
```

Alamprogramm: argumendid ja skoop

```
def rida(nimi, hind, kogus=1):          # kogus on vaikeväärtusega
    summa = hind * kogus                 # summa on lokaalne muutuja
    return f"{nimi:<12}{summa:>8.2f}"

print(rida("Piim", 1.29, kogus=2))      # nimeline argument
```

R: funktsiooni sees loodud muutuja ei ole väljaspool nähtav. K: ära muuda globaalseid muutujaid funktsiooni seest; anna andmed argumendina ja tagasta tulemus.

Andmevahetus kasutajaga

Pseudokood	Python
`SISESTA nimi`	`nimi = input("Nimi: ")`
`SISESTA täisarv n`	`n = int(input("n: "))`
`SISESTA arv x`	`x = float(input("x: ").replace(",", "."))`

Pseudokood	Python
`VÄLJASTA x kahe komakohaga`	`print(f"{x:.2f}")`
`VÄLJASTA tabelina`	`print(f"{nimi:<10}{hind:>8.2f}")`

Andmevahetus failidega

```
LOE FAILIST "hinnad.txt"
    IGA rida FAILIS KORDA
        nimi, hind ← tükelda rida ";"-ga
KIRJUTA FAILI "arve.txt"
    KIRJUTA "Kokku: " + summa
```

```
# tekstifail rida-realt
with open("hinnad.txt", encoding="utf-8") as fail:
    for rida in fail:
        nimi, hind = rida.strip().split(";")

with open("arve.txt", "w", encoding="utf-8") as fail:
    fail.write(f"Kokku: {summa:.2f}\n")

# CSV (Exceli eksport: semikoolon ja BOM!)
import csv
with open("tooted.csv", encoding="utf-8-sig", newline="") as fail:
    for kirje in csv.DictReader(fail, delimiter=";"):
        print(kirje["nimi"], kirje["hind"])

# JSON
import json
with open("ostukorv.json", encoding="utf-8") as fail:
    ostukorv = json.load(fail)
with open("ostukorv.json", "w", encoding="utf-8") as fail:
    json.dump(ostukorv, fail, ensure_ascii=False, indent=2)
```

R: `with` sulgeb faili ka vea korral. K: anna alati `encoding="utf-8"`, muidu võivad täpitähed (õ, ä, ö, ü) Windowsis katki minna.

ÕV4 – Andmetüübid ja -struktuurid

Tunneb põhilisi andmetüüpe ja -struktuure (täis- ja ujukomaarvud, tõeväärtused, sõned, järjendid, hulgad, sõnastikud, ennikud) ning oskab kasutada vastavaid standardoperatsioone.

Tüüp	Literaal	Muudetav?	Järjestatud?	Millal kasutada
`int`	`3`, `-7`	ei	–	loendamine, kogused
`float`	`12.5`, `1e-3`	ei	–	mõõtmised, hinnad (NB! `0.1 + 0.2 != 0.3`)
`bool`	`True`, `False`	ei	–	tingimuste tulemused, lipud
`str`	`"piim"`	ei	jah	tekst
`list` (järjend)	`[1, 2, 3]`	jah	jah	muutuv jada
`tuple` (ennik)	`(59.4, 24.7)`	ei	jah	fikseeritud kirje, nt koordinaat
`set` (hulk)	`{"a", "b"}`	jah	ei	unikaalsus, kuuluvuse kontroll
`dict` (sõnastik)	`{"piim": 1.29}`	jah	sisestusjärjekord	võti → väärtus otsing

Arvud ja tüübiteisendus

```
int("42"), float("12.5"), str(3.0)   # 42, 12.5, "3.0"
round(2.675, 2)                      # 2.67 – ujukoma eripära
7 / 2, 7 // 2, 7 % 2                 # 3.5, 3, 1
bool(0), bool(""), bool([])          # kõik False
```

Sõned

```
s = "Tere, Tartu!"
s[0], s[-1], s[0:4], s[::-1]          # "T", "!", "Tere", "!utraT ,ereT"
s.lower(), s.upper(), s.strip()
s.split(", "), ";".join(["a", "b"])  # ["Tere", "Tartu!"], "a;b"
s.replace("Tartu", "Tallinn"), s.find("T"), s.count("e")
"tar" in s.lower(), s.startswith("Tere")
len(s), ord("õ"), chr(245)           # 12, 245, "õ"
```

Järjendid ja ennikud

```
hinnad = [3.5, 1.2, 8.0]
hinnad.append(2.0); hinnad.insert(0, 9.9); hinnad.remove(1.2); hinnad.pop()
sorted(hinnad, reverse=True)          # uus järjend
hinnad.sort()                         # muudab kohapeal, tagastab None
min(hinnad), max(hinnad), sum(hinnad), len(hinnad)
esimene, *ylejaanud = hinnad          # lahtipakkimine

punkt = (59.4, 24.7)                  # ennik: muuta ei saa
laius, pikkus = punkt
```

R: järjend on muudetav objekt – `b = a` ei tee koopiat, mõlemad nimed viitavad samale järjendile. Koopia: `b = a.copy()` või `b = a[:]`.

Hulgad

```
varem = {"piim", "leib"}
nüüd = {"leib", "õun"}
varem | nüüd, varem & nüüd, nüüd - varem   # ühend, ühisosa, vahe
unikaalsed = set(ostud)
```

Sõnastikud

```
ladu = {"piim": 12, "leib": 5}
ladu["õun"] = 30                      # lisa / muuda
ladu.get("juust", 0)                  # 0, kui võtit pole
for toode, kogus in ladu.items():
    print(toode, kogus)
"piim" in ladu                        # True
del ladu["leib"]                      # eemalda võti
```

Rühmitamine ja comprehensionid

```
loendur ← tühi sõnastik
IGA toode KOGUMIS ostud KORDA
    loendur[toode] ← loendur[toode] (või 0) + 1
LÕPP IGA
```

```
loendur = {}
for toode in ostud:
    loendur[toode] = loendur.get(toode, 0) + 1

ruudud = [x * x for x in range(5)]                    # järjend
kallid = [h for h in hinnad if h > 5]                 # filtreerimine
km_ga = {t: h * 1.24 for t, h in hinnakiri.items()}   # sõnastik
tahed = {s[0] for s in nimed}                         # hulk
```

NB! `sorted()` järjestab Unicode'i koodi järgi, mitte eesti tähestiku järgi (õ, ä, ö, ü, š, ž satuvad valesse kohta).

ÕV5 – Programmi töö käigu analüüs ja laiendamine

Oskab analüüsida ja üksikasjalikult selgitada programmi töö käiku ning programmi laiendada.

Töö käiku selgitad jälitustabeliga: üks rida iga täidetud sammu kohta, üks veerg iga muutuja kohta. Thonnys teeb sama "Step into" (F7) ja muutujate paneel.

```
hinnad = [4, 12, 7]
summa = 0
kalleim = hinnad[0]
for h in hinnad:
    summa += h
    if h > kalleim:
        kalleim = h
print(summa, kalleim)
```

Samm	`h`	`summa`	`kalleim`	`h > kalleim` ?
algus	–	0	4	–
1. kordus	4	4	4	False

Samm	`h`	`summa`	`kalleim`	`h > kalleim` ?
2. kordus	12	16	12	True
3. kordus	7	23	12	False
`print`	–	23	12	–

Selgitamise mall (suuline eksam):

1. **Mida programm teeb?** – üks lause: "leiab hindade summa ja suurima hinna".
2. **Millised on sisend ja väljund?** – järjend `hinnad` → kaks arvu.
3. **Mis rida mida teeb?** – algväärtustamine, tsükkel, kogumine, võrdlus.
4. **Mis juhtub piirjuhul?** – tühi järjend → `hinnad[0]` annab `IndexError`.
5. **Kuidas seda parandada või laiendada?**

Laiendamine: samm-sammult, üks muudatus korraga

```
Laiendus: leia ka kalleima toote nimi.
1. Muuda andmestruktuuri: hinnad → sõnastik toode → hind
2. Muuda tsüklit: IGA (toode, hind) KOGUMIS hinnakiri
3. Lisa muutuja kalleim_toode
4. Kaitse piirjuht: KUI hinnakiri on tühi SIIS TAGASTA (0, puudub)
5. Testi: vana käitumine (summa) peab jääma samaks
```

```
def kokkuvote(hinnakiri):
    if not hinnakiri:
        return 0, None
    summa = sum(hinnakiri.values())
    kalleim_toode = max(hinnakiri, key=hinnakiri.get)
    return summa, kalleim_toode

print(kokkuvote({"leib": 4, "juust": 12, "õun": 7}))   # (23, 'juust')
```

K: enne laiendamist refaktoreeri kood funktsiooniks; siis saab vana ja uut käitumist testiga võrrelda.

ÕV6 – Algoritm, vormistus, silumine ja testimine

Oskab luua lihtsamat ülesannet lahendava algoritmi, koostada ja korrektselt vormistada lahendusprogrammi ning seda siluda ja testida.

Töövoog on alati sama: algoritm pseudokoodina → programm → vormistus → testid → silumine, kuni testid läbivad.

Algoritm: lineaarotsing

```
FUNKTSIOON leia_hind(hinnakiri, otsitav)
    IGA (nimi, hind) KOGUMIS hinnakiri KORDA
        KUI nimi = otsitav SIIS TAGASTA hind
    LÕPP IGA
    TAGASTA puudub
LÕPP FUNKTSIOON
```

```
def leia_hind(hinnakiri, otsitav):
    """Tagastab toote hinna või None, kui toodet pole."""
    for nimi, hind in hinnakiri:
        if nimi == otsitav:
            return hind
    return None
```

Korrektne vormistus

```
"""hinnaarvestus.py – ostukorvi summa arvutamine."""

import json                          # 1. impordid faili alguses

KAIBEMAKSU_MAAR = 0.24               # 2. nimega konstandid


def arvuta_summa(ostukorv):          # 3. funktsioonid docstringiga
    """Tagastab ostukorvi summa koos käibemaksuga."""
    return sum(h * k for h, k in ostukorv) * (1 + KAIBEMAKSU_MAAR)


def main():                          # 4. põhiprogramm
    ...


if __name__ == "__main__":           # 5. käivitub ainult otse jooksutades
    main()
```

Kontroll käsurealt: `ruff check .` (vead, stiil) ja `ruff format .` (vormindus). K: funktsioonide vahel 2 tühja rida, rea pikkus kuni 88 märki.

Silumine

1. **Loe veateadet altpoolt üles:** viimane rida ütleb vea tüübi, sellest ülemine rida faili ja reanumbri.
2. **Taasta viga väikseima sisendiga.**
3. **Vaata muutujate väärtusi:** Thonny Debug (Ctrl+F5) või ajutine `print(f"{summa=}")`.
4. **Paranda üks asi, jooksuta testid uuesti.**

Veateade	Tüüpiline põhjus
`SyntaxError`	puudu koolon, sulg või jutumärk
`IndentationError`	taane vale või segamini tabid ja tühikud
`NameError`	muutuja nimes trükiviga või kasutatakse enne omistamist
`TypeError`	`"5" + 3` – unustati `int()` / `float()`
`ValueError`	`float("12,5")` – koma punkti asemel
`IndexError`	indeks ⩾ `len(järjend)` või tühi järjend
`KeyError`	sõnastikus pole sellist võtit – kasuta `.get()`
`ZeroDivisionError`	jagaja on 0 – kontrolli piirjuhtu
`FileNotFoundError`	vale failitee või töökaust

Testimine pytestiga

```
Iga funktsiooni kohta testi:
    1. tavaline juht
    2. piirjuht (0, tühi, täpselt piiril)
    3. vigane sisend (oodatav erind)
```

```
# test_hinnaarvestus.py
import pytest
from hinnaarvestus import arvuta_summa, leia_hind
```

```
def test_tavaline():
    assert arvuta_summa([(10, 2)]) == pytest.approx(24.8)


def test_tyhi_ostukorv():
    assert arvuta_summa([]) == 0


@pytest.mark.parametrize("otsitav, oodatud", [
    ("piim", 1.29),
    ("leib", None),
])
def test_leia_hind(otsitav, oodatud):
    assert leia_hind([("piim", 1.29)], otsitav) == oodatud


def test_vigane_sisend():
    with pytest.raises(TypeError):
        arvuta_summa([("10", 2)])
```

Käivitus: `pytest` (või `uv run pytest`). R: testifaili ja -funktsiooni nimi algab `test_`. K: ujukomaarve võrdle `pytest.approx` abil, mitte `==`.

Õpiväljundid ajakavas

Iga ajakava teema toetab üht või mitut õpiväljundit; tabel näitab, millise spikri jaotisega teemat korrata.

Kuupäev	Teema	Õpiväljund	Spikri jaotis
08.09.2026	Algoritm ja programm	ÕV1, ÕV2, ÕV6	Kokkulepped; ÕV1 jada
08.09.2026	Töökeskkonna paigaldus. Versioonihaldus	ÕV6	ÕV6 vormistus
15.09.2026	Andmetüübid ja muutujad. Avaldised ja tehete järjekord	ÕV3, ÕV4	ÕV3 muutuja/avaldis; ÕV4 tabel
15.09.2026	Tüübiteisendused, vormindus, nimega konstandid	ÕV2, ÕV3, ÕV4	ÕV2 näidis; ÕV3 kasutajaga; ÕV4 arvud

Kuupäev	Teema	Õpiväljund	Spikri jaotis
22.09.2026	Loogilised tehted. Mitmeosaline ja mitmeharuline tingimuslause	ÕV1, ÕV3	ÕV1 hargnemine; ÕV3 tingimuslause
22.09.2026	Tingimuslaused ja piirjuhtude kontroll	ÕV3, ÕV6	ÕV3 piirjuhud
29.09.2026	Tsüklid ja iteratsioon	ÕV1, ÕV3	ÕV1 kordus; ÕV3 tsükkel
29.09.2026	Tsüklite koostamine	ÕV2, ÕV5	ÕV5 jälitustabel
06.10.2026	Sõned. Indekseerimine, viilutamine, märgikodeering	ÕV4	ÕV4 sõned
06.10.2026	Sõnemeetodid ja tekstitöötlus	ÕV2, ÕV4	ÕV4 sõned
13.10.2026	Järjendid ja ennikud. Muudetavus	ÕV4	ÕV4 järjendid ja ennikud
13.10.2026	Järjendioperatsioonid, lahtipakkimine, sortimine	ÕV4	ÕV4 järjendid ja ennikud
20.10.2026	Rühmitamine ja andmestruktuuride koostamine	ÕV4, ÕV5	ÕV4 hulgad, sõnastikud, comprehensionid
27.10.2026	Funktsioon kui alamprogramm. Argumendid ja skoop	ÕV1, ÕV3	ÕV1 alamprogramm; ÕV3 skoop
27.10.2026	Funktsioonide kirjutamine ja refaktoreerimine	ÕV2, ÕV5	ÕV5 laiendamine
03.11.2026	Silumine. Testimise põhimõtted ja piirjuhud. pytest	ÕV5, ÕV6	ÕV6 silumine ja testimine
03.11.2026	Teekide kasutamine. Projekti struktuur ja dokumenteerimine	ÕV6	ÕV6 vormistus
10.11.2026	Failid ja tekstiandmed. CSV ja JSON	ÕV3	ÕV3 failidega
10.11.2026	API-päringud ja erindite käsitlemine	ÕV2, ÕV3	ÕV2 `loe_arv`; ÕV6 veateadete tabel